Chorale Coder — Engineering Benchmark Report

A tiered, execution-graded benchmark of three models on real software engineering — from multi-file libraries and debugging up to an end-to-end, multi-layered full-stack web application.

Models: Gemma-4-31B (HF) · gpt-oss-120B (Fireworks) · MiniMax-M2.7 (Fireworks) · Graded by running the code, with partial credit · Tier 1 N=1, Tier 2 N=3 trials

---

TL;DR

- Two difficulty tiers: **Tier 1** — four everyday tasks (build a library, debug a broken codebase, async concurrency, a stateful CLI); **Tier 2** — three hard tasks (a mini web framework, a Redux-like store, and an **end-to-end full-stack app**: HTTP server + REST API + JSON persistence + served frontend).
- **On easy-to-moderate work, all three are excellent** and Gemma-4-31B wins on value (fastest, free). **On hard, multi-layered work, reliability separates them** — and that is where the frontier model earns its keep.
- **gpt-oss-120B is the reliability champion: a perfect score on every Tier-2 task across all three trials** — including the full-stack app 3/3.
- **Gemma-4-31B is fast and often correct but inconsistent on the full-stack task** — its server failed to start in **2 of 3 trials** (it hardcodes the port instead of honoring `PORT`). Fast and free, but not yet trustworthy for complex apps unattended.
- **This validates the escalation architecture with evidence:** default to Gemma-4-31B for routine work; escalate to gpt-oss-120B for complex, multi-layered, or critical builds.

---

1. Why This Benchmark

An earlier evaluation ranked eleven models on single-file algorithmic *puzzles*. Its central caveat: puzzles are not engineering, and the frontier models might justify their cost on real, multi-file, tool-driven work. A first real-engineering pass (Tier 1 below) tested that and found the value king held up. **This report**

ramps the difficulty further — culminating in a full-stack application — to find where, if anywhere, the picture changes. It does change, and the change is instructive.

Every task runs through Chorale's **production coder agent with its full toolset**, including `bash` — so the model reads files, writes multiple files, runs tests, and iterates like a developer. Nothing is mocked.

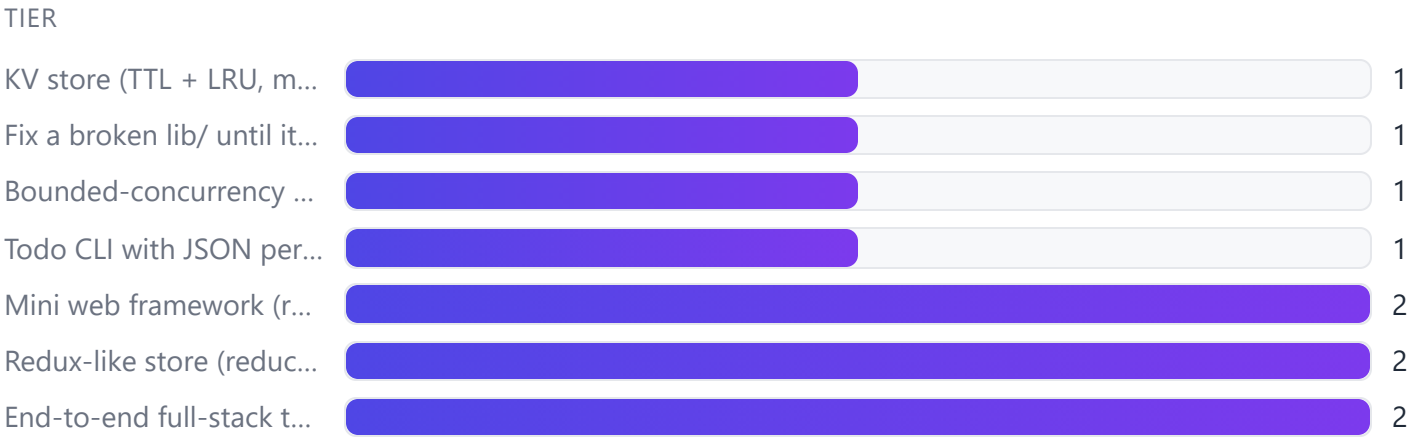
2. Methodology

- **Execution-graded, partial credit.** Each project is scored by *running* the result against hidden checks — a library is imported and exercised, a debug target's own suite is run, a framework is driven through `inject`, and the **full-stack server is actually spawned and hit over HTTP** (GET/POST/PATCH/DELETE, status codes, content types, persistence). Score = checks passed / total.
- **Graders validated first.** Every grader was checked against known-good *and* known-bad reference solutions before any model ran (`eval/projects-selftest.ts`), catching two harness bugs (CLI state leakage; a missing per-check timeout that a non-responding route could exploit) before they could bias results.
- **Trials.** Tier 1 is reported at N=1 (its tasks proved stable). Tier 2 is reported at **N=3** because the harder tasks showed real run-to-run variance — the single most important methodological point in this report.
- **Cost** is normalized (cached input + output at Fireworks list rates); Gemma runs on HF (≈$0 per the billing delta).

3. The Tasks

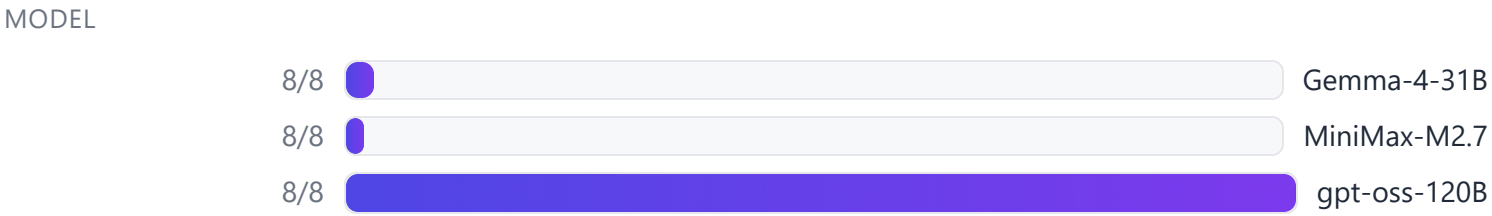
Tier	Project	Skill	Checks
1	KV store (TTL + LRU, multi-file)	Build a library	8
1	Fix a broken `lib/` until its suite passes	Debug a codebase	7
1	Bounded-concurrency promise pool	Async correctness	5
1	Todo CLI with JSON persistence	Integration / I·O	6
2	Mini web framework (routing, params, middleware, `inject`)	Framework design	7

Tier	Project	Skill	Checks
2	Redux-like store (reducers, subscribe, middleware, `combineReducers`)	State architecture	7
2	**End-to-end full-stack task app** (HTTP server + REST + persistence + UI)	Full-stack	10



4. Tier 1 — Everyday Engineering (N=1)

Model	KV	Debug	Async	CLI	Overall	Time	Cost
Gemma-4-31B	8/8	7/7	5/5	6/6	**26/26 · 100%**	**32s**	**≈$0.00**
MiniMax-M2.7	8/8	7/7	5/5	6/6	**26/26 · 100%**	73s	$0.0246
gpt-oss-120B	8/8	7/7	4/5	6/6	25/26 · 96%	68s	$0.0071



At this level the value king dominates: Gemma-4-31B is perfect, ~2× faster than either frontier model, and free. gpt-oss-120B's only slip was a promise pool that didn't reject on a task error. **On routine engineering, cheaper-and-faster wins.**

5. Tier 2 — Advanced Engineering (N=3 trials)

Aggregated over three independent trials (checks passed / checks attempted):

Model	Framework	Store	Full-stack	Overall	Avg time/trial	Avg cost/trial
gpt-oss-120B	21/21	21/21	**30/30**	**72/72 · 100%**	117s	$0.0096
MiniMax-M2.7	17/21	21/21	30/30	68/72 · 94%	99s	$0.0380
Gemma-4-31B	20/21	21/21	**10/30**	51/72 · 71%	**25s**	**≈$0.00**

Reliability — did each model fully pass a task, per trial:

Task	gpt-oss-120B	MiniMax-M2.7	Gemma-4-31B
Framework (7/7)	3/3	2/3	2/3
Store (7/7)	3/3	3/3	3/3
Full-stack (10/10)	**3/3**	**3/3**	**1/3**

The story the aggregate tells:

- **gpt-oss-120B — flawless and consistent.** Every Tier-2 task, every trial: 100%. It is slow (~117s/trial) and no longer free, but on hard work it simply does not miss.
- **MiniMax-M2.7 — nearly there.** Full-stack 3/3 and store 3/3, but one trial produced a broken framework (`inject` returned a malformed status), dropping it to 94%. Reliable on the app, occasionally shaky on abstractions. Also the most expensive (~4× gpt-oss).
- **Gemma-4-31B — fast, free, but not yet dependable on the hardest task.** Framework and store are near-perfect, but the **full-stack server started successfully in only 1 of 3 trials**. In the two failures the server never bound to the assigned port — Gemma hardcoded `3000` instead of reading `process.env.PORT` , a concrete, repeatable spec-compliance gap.

6. Deep-Dive — the Full-Stack Application

The centerpiece task asks for a complete, runnable web app in Node built-ins only: a REST API (`GET/POST/PATCH/DELETE /api/tasks` with correct status codes and `application/json`), a `tasks.json` persistence layer, an HTML+JS frontend served at `/` that fetches and renders the API, and 404s for unknown routes. The grader **spawns the server on a fresh port and drives all ten behaviors over real HTTP**, then kills it.

- **gpt-oss-120B and MiniMax-M2.7 delivered a correct, fully working full-stack app in every trial** — routing, REST semantics, body parsing, content types, persistence, and an integrated frontend all correct.
- **Gemma-4-31B produced correct-looking code but a server that only ran when the port matched its hardcoded default.** The application logic was often right; the *operational contract* (honor the injected port, actually come up) was not. For an agent expected to build and run apps unattended, that is the difference between "works" and "works on my machine."

This is exactly the failure mode the puzzle and Tier-1 benchmarks could not surface: correctness of *logic* is necessary but not sufficient for real full-stack work; the *operational and integration details* are where a fast, cheap model quietly slips.

7. Analysis — the Difficulty Crossover

The two tiers tell opposite stories, and both are true:

- **On easy-to-moderate engineering, value wins.** Gemma-4-31B is perfect, fastest, and free (Tier 1: 100%). Paying 10–40× for a frontier model buys nothing.

- **On hard, multi-layered engineering, reliability wins.** The full-stack app flips the ranking: gpt-oss-120B (100%, 3/3) and MiniMax-M2.7 (100%, 3/3) are dependable; Gemma (33% of full-stack checks, 1/3 trials) is not. This is the first place in the entire evaluation series where the frontier models clearly and repeatably earn their cost.

The crossover is the whole point. A single-model policy is wrong in one direction or the other. The right policy is **tiered routing**.

8. The Decision

> **Default: Gemma-4-31B → Escalate: gpt-oss-120B** — now evidence-backed on both ends.

- **Gemma-4-31B for routine and moderate work** — libraries, debugging, refactors, CLIs, algorithmic logic. Perfect at Tier 1, ~2× faster, free. The overwhelming majority of coder turns.
- **gpt-oss-120B for complex, multi-layered, or must-work-first-time builds** — full-stack apps, servers, anything where the operational contract matters. It was the only model to go 100% across every Tier-2 task and trial. This is precisely the "hard case" the escalation exists for — and the full-stack results show Gemma's default *does* fail there, so the escalation is not theoretical.
- **MiniMax-M2.7** is a capable reliability-first alternative (full-stack 3/3) but costs ~4× gpt-oss and slipped once on the framework — no reason to prefer it over gpt-oss-120B here.

9. Limitations & Honest Caveats

1. **Tier 2 is N=3; Tier 1 is N=1.** Three trials expose variance but do not pin exact rates — read "1/3" and "3/3" as *directional reliability*, not precise probabilities. Gemma's full-stack failures were the same root cause (port handling) twice, which raises confidence that it is a real, repeatable weakness rather than noise.
2. **Still bounded projects.** Real *engineering* (multi-file, stateful, tool-driven, a running server) but small and self-contained — not thousand-line codebases, ambiguous specs, or long-horizon feature work. The trends are clear at this scale; larger tasks remain future work.
3. **Graders validated, not infallible.** Every grader passed reference good/bad solutions before the run. Two harness bugs were found and fixed pre-run; others may remain.
4. **Normalized cost.** Recomputed from measured tokens × list rates (cached input + output); real bills vary with cache-hit rate. Gemma runs on HF (≈$0).

10. Reproducibility

```
# Validate every grader against reference good/bad solutions
npx tsx eval/projects-selftest.ts

# Tier 1 (four everyday tasks)
npx tsx eval/coder-projects.ts --only=kvstore,debug,asyncpool,cli

# Tier 2 (framework, store, full-stack) — run several times for reliability
npx tsx eval/coder-projects.ts --only=framework,store,fullstack
```

Harness: `eval/coder-projects.ts` · Grader self-test: `eval/projects-selftest.ts` · Reference solutions: `eval/projects/_ref/` · Tier-1 detail: `eval/PROJECTS-RESULTS.md`

---

11. Appendix — Can Prompting Fix Gemma's Full-Stack Lapse?

Gemma's full-stack failures all shared one root cause: it hardcoded the port instead of reading `process.env.PORT`, so the grader's spawned server never came up. Two mechanisms could plausibly address it — **content-level salvage** (post-process the output) or **meta-prompting** (steer the model). Salvage was rejected as the wrong tool: a regex that rewrites `.listen(3000)` is brittle (misses `const P = 3000` and framework variants), risky (fixed ports are sometimes intentional), and would overfit to this one benchmark. Meta-prompting was tested properly.

A general **operational-correctness rule** was added to `agents/coder.md` (honor the interface contract literally; never hardcode a value the task said to make configurable; read ports/paths from the environment; re-check the contract before finishing) — general craft guidance that applies to every model and task, not a patch aimed at the port bug. Then the full-stack task was run **10× before and 10× after** (Gemma is free on HF):

Condition	Full-pass	Mean	Failure mode
Baseline (current prompt)	7/10	70%	hardcoded port × 3
+ operational-correctness rule	8/10	80%	hardcoded port × 2

FULL-PASS

A one-run difference over ten is within noise — not evidence of a real fix. Even with a rule that explicitly says "read `process.env.PORT`, never hardcode," *and* the task prompt already stating the same, Gemma still hardcoded the port in ~20% of runs. This is an instruction-*following* lapse, and prompting nudges it at best. The rule was kept — it is sound general guidance that can only help across operational tasks — but the honest conclusion is that **neither salvage nor meta-prompting makes Gemma reliable for unattended full-stack work.** The dependable fix is the architecture already in place: **escalate to gpt-oss-120B for complex/critical builds** (a perfect record here). This experiment confirms that decision rather than replacing it.

---

12. Few-Shot & Self-Healing — Two More Levers, Measured

Following the prompting experiment, two further mechanisms were built as **customizable, default-on tick-boxes** on the coder (`fewShot`, `selfHeal`; a third, `selfLearn`, is declared but parked for Phase 3 — see `docs/ROADMAP.md`).

Few-shot (`fewShot`) injects worked operational patterns (`agents/coder.examples.md`: a server reading `process.env.PORT`, a CLI parsing argv, correct REST status codes) — *show* the pattern instead of *stating* the rule. Gemma full-stack, 10× each:

Condition	Full-pass	Mean
Baseline (no rule)	7/10	70%
+ operational rule	8/10	80%
+ few-shot examples	8/10	80%

No measurable gain over the rule — showing did not beat telling for this lapse.

Self-healing (`selfHeal`) goes beyond syntax verification: it **runs** what the model wrote — smoke-boots a written server on an injected `PORT` and probes that port, and smoke-imports modules to catch "compiles but throws on load" — feeding any runtime failure back into the repair loop. It is **deterministically validated** (`eval/smoke-selftest.ts`): it flags a hardcoded-port server and a crash-on-load module, and passes correct ones.

But the live runs delivered a **diagnostic surprise**: across 20 Gemma full-stack runs, self-heal logged **zero catches**. Inspecting the failures showed *why* — Gemma's full-stack failures are overwhelmingly **template-literal syntax errors** (`Syntax error "\ "`) in the large inline HTML string, which crash the server before it can run. Those are caught by *syntax* verification (self-heal only runs on syntactically-clean code), and Gemma frequently cannot repair them within the round budget. The "server did not start" failures were mostly malformed HTML templates, **not** hardcoded ports. With all mechanisms on, one 10-run sample reached 10/10; isolating self-heal (few-shot off) gave 8/10.

Interim conclusion. Both mechanisms are sound, general infrastructure — self-heal in particular is a *proven ground-truth safety net* for the "logically right, operationally wrong" class, and it benefits **every** model. But at this point neither had made Gemma reliable for complex full-stack work, and its dominant failure looked like an immovable **capability limit**. That framing turned out to be wrong once the failure was diagnosed precisely — **see §13.** (Self-learning — reflect on failures, reuse lessons and self-derived exemplars — remains the next lever, parked for Phase 3.)

---

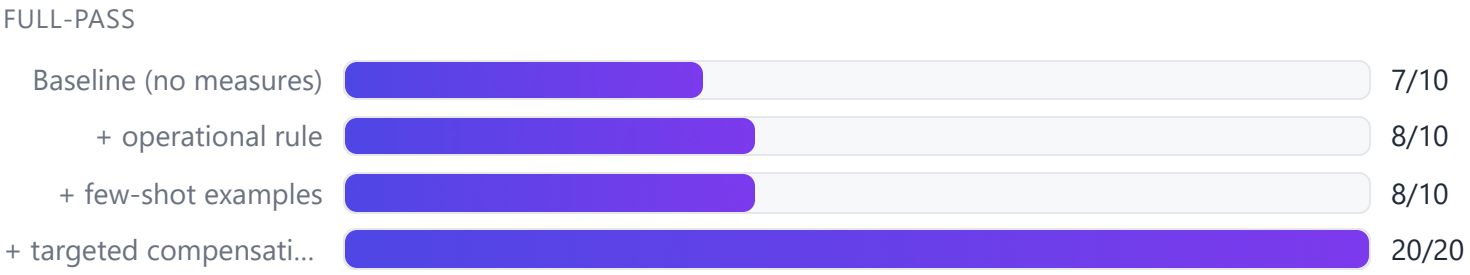
13. Compensating for the Weakness — a Measured Fix

The coder's mandate is to *compensate for each model's shortcomings*, not route around them. §12 stopped one step short: it diagnosed Gemma's failure (large HTML embedded in a JS template literal → a nested backtick closes the string early → `Syntax error "\ "`) but concluded escalation was the fix. With the cause pinned down, three **targeted, general** measures were added instead:

1. **Structural steer (few-shot).** A `coder.examples.md` exemplar shows HTML served from a separate `.html` file via `readFileSync` — removing the nested-backtick trap entirely instead of asking the model to navigate it.
2. **Targeted repair diagnostics.** When syntax verification sees a backtick / template-literal error, the feedback now *names the exact cause* (a big HTML page with an inline `<script>` nested in a JS template literal) and *prescribes the file-based fix* — rather than a generic "rewrite it."
3. **More repair headroom.** The verify-repair budget was raised from 3 to 5 rounds so a full restructure fits.

None of these is grader-specific — they are general craft that helps every model and any HTML-serving task. Measured on Gemma full-stack, **20 runs (two batches of 10)**:

Condition	Full-pass	Mean
Baseline (no measures)	7/10	70%
+ operational rule	8/10	80%
+ few-shot examples	8/10	80%
+ targeted compensation (steer + diagnostics + budget)	**20/20**	**100%**



Gemma went from ~70% to a clean **100% across 20 runs**, most **one-shot** — servers now start in 3–6s, and the 16–18s "server did not start" stall signature is gone.

The lesson generalizes the coder's purpose. A precisely diagnosed model weakness *can* be engineered around from our end — by changing the strategy to one the model executes reliably, and by making the repair loop's feedback specific to the failure rather than generic. Escalation to gpt-oss-120B stays available for genuinely hard cases, but for this class the coder now **compensates**, exactly as intended — which makes Gemma-4-31B a viable default for full-stack projects, not just simple ones.

Chorale is an open-source, model-agnostic multi-agent system. Measurements were taken through its production coder pipeline; numbers will shift as models, prices, and the pipeline evolve.